Vel Tech Group of Institutions

Name: KISHORE P P
Email: vtu27958@veltech.edu.in
Roll no: vtu27958
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS5L14

VTU_DAA_Week 8_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 21

Section 1 : Coding

1. Problem Statement

Ram is tasked with finding the shortest distances from a source router to all other routers in a network. The network is represented as a graph of N routers connected by M bidirectional weighted links.

The program uses Dijkstra's Algorithm: starting from the source router, it repeatedly selects the unvisited router with the smallest known distance, then updates distances to its neighbors if a shorter path is found through it. This continues until all routers have been visited.

Given the network details and a source router, find and print the shortest distance from the source router to every router in the network.

Input Format

The first line of input consists of an integer N, representing the number of routers.

The second line of input consists of an integer M, representing the number of links.

The next M lines each consist of three space-separated integers: r1, r2, and w, representing a bidirectional link between router r1 and router r2 with weight w.

The last line of input consists of an integer s, representing the source router.

Output Format

The output consists of N lines printed in ascending order of router index (from 0 to N-1).

Each line prints two space-separated integers: the router index and its shortest distance from the source router.

The source router itself is printed with distance 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
5
0 1 1
0 2 3
1 2 1
1 3 4
2 3 1
0
Output: 0 0
1 1
2 2
3 3

Answer

```
#include <stdio.h>
#include <limits.h>

#define MAX_ROUTERS 10
```

```
void dijkstra(int graph[MAX_ROUTERS][MAX_ROUTERS], int adj[MAX_ROUTERS]
[MAX_ROUTERS], int src, int n) {
    int dist[MAX_ROUTERS];
    int visited[MAX_ROUTERS];

    for (int i = 0; i < n; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }

    dist[src] = 0;

    for (int count = 0; count < n; count++) {
        int min = INT_MAX, u = -1;

        for (int v = 0; v < n; v++) {
            if (!visited[v] && dist[v] <= min) {
                min = dist[v];
                u = v;
            }
        }

        if (u == -1) break;

        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (!visited[v] && graph[u][v] != 0 && dist[u] != INT_MAX
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    for (int i = 0; i < n; i++) {
        printf("%d %d\n", i, dist[i]);
    }
}

int main_logic() {
    return 0;
```

```
}

int main() {
    int numRouters, numLinks;
    scanf("%d", &numRouters);
    scanf("%d", &numLinks);
    int graph[MAX_ROUTERS][MAX_ROUTERS] = {0};
    int adj[MAX_ROUTERS][MAX_ROUTERS] = {0};
    for (int i = 0; i < numLinks; i++) {
        int router1, router2, weight;
        scanf("%d %d %d", &router1, &router2, &weight);
        graph[router1][router2] = weight;
        graph[router2][router1] = weight;
        adj[router1][router2] = 1;
        adj[router2][router1] = 1;
    }
    int source;
    scanf("%d", &source);
    dijkstra(graph, adj, source, numRouters);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Alice is working on a delivery system for an e-commerce platform. The system uses a network of delivery hubs connected by roads, each with a certain travel time in minutes.

Alice needs to find the fastest delivery time from a source hub to every other hub in the network. Given the network details and a source hub, the program uses Dijkstra's Algorithm to find and print the shortest delivery time from the source hub to all hubs in the network.

Dijkstra's Algorithm works by starting from the source hub with distance 0, then repeatedly selecting the unvisited hub with the smallest known time and updating the times to its neighbors if a shorter path is found through it. This continues until all hubs have been visited.

Input Format

The first line of input consists of an integer N, representing the number of delivery hubs.

The second line of input consists of an integer M, representing the number of roads between hubs.

The next M lines each consist of three space-separated integers: r1, r2, and t, representing a bidirectional road between hub r1 and hub r2 with a delivery time of t minutes.

The last line of input consists of an integer s, representing the source delivery hub.

Output Format

The output prints N lines representing the shortest delivery times from the source hub to all hubs (including the source) in ascending order of hub index.

Each line prints two space-separated integers: the hub index and its shortest delivery time from the source hub.

The delivery time of the source hub itself is 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
3
0 1 2
0 2 4
1 2 1
0
Output: 0 0
1 2
2 3

Answer

```
#include <stdio.h>
```

```
#include <limits.h>

#define MAX_NODES 10

int graph[MAX_NODES][MAX_NODES];
int dist[MAX_NODES]; // Declared globally so the Footer's main can access it

void dijkstra(int n, int src) {
    int visited[MAX_NODES];

    for (int i = 0; i < n; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }

    dist[src] = 0;

    for (int count = 0; count < n; count++) {
        int min = INT_MAX, u = -1;

        for (int v = 0; v < n; v++) {
            if (!visited[v] && dist[v] <= min) {
                min = dist[v];
                u = v;
            }
        }

        if (u == -1 || dist[u] == INT_MAX) break;

        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (!visited[v] && graph[u][v] != -1) {
                if (dist[u] + graph[u][v] < dist[v]) {
                    dist[v] = dist[u] + graph[u][v];
                }
            }
        }
    }
    // Note: Do not put printf here if the Footer's main already contains a print
loop.
    // Based on your error, the Footer has its own printf("%d %d\n", i, dist[i]);
}
```

```
int main() {
    int N, M, s;
    scanf("%d", &N);
    scanf("%d", &M);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            graph[i][j] = -1;
        }
    }
    for (int i = 0; i < M; i++) {
        int r1, r2, w;
        scanf("%d %d %d", &r1, &r2, &w);
        graph[r1][r2] = w;
        graph[r2][r1] = w;
    }
    scanf("%d", &s);
    dijkstra(N, s);
    for (int i = 0; i < N; i++) {
        printf("%d %d\n", i, dist[i]);
    }
    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem statement

Create a program to calculate and display the topological order of a Directed Acyclic Graph (DAG) with n vertices. The graph is represented as a lower triangular adjacency matrix, where for every vertex i and vertex j with j < i, the edge from i to j exists (value 1), and no edge exists from j to i (value 0). All diagonal and upper triangular values are 0.

The program should:

Construct and display the adjacency matrix of the graph.Perform a Depth First Search (DFS) traversal on the graph.Display the topological order of the vertices in reverse order of their DFS end times.

Input Format

The first line of input consists of an integer n, representing the number of

vertices in the graph. No further input is required; the adjacency matrix is constructed internally by the program.

Output Format

he first line of output prints: Adjacency Matrix of the graph

The next n lines print the adjacency matrix row by row, where each value is preceded by a tab character (\t).

A blank line is printed after the matrix.

The next line prints: Topological order:

The final line prints the topological order of the vertices in the format: -->v1-->v2-->...-->vn, where vertices appear in reverse order of their DFS end times.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

Output: Adjacency Matrix of the graph

```
 0  0  0
 1  0  0
 1  1  0
```

Topological order:
-->2-->1-->0

Answer

```
#include<stdio.h>

int s[100], res[100], top;

void buildMatrix(int a[100][100], int n) {
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         if (i > j) a[i][j] = 1;
         else a[i][j] = 0;
      }
```

```
    }
}

void dfs(int n, int a[100][100], int u, int visited[]) {
    visited[u] = 1;
    for (int v = 0; v < n; v++) {
        if (a[u][v] == 1 && !visited[v]) {
            dfs(n, a, v, visited);
        }
    }
    res[top++] = u;
}

void topological_order(int n, int a[100][100]) {
    int visited[100] = {0};
    top = 0;
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs(n, a, i, visited);
        }
    }
    printf("\nTopological order:\n");
    for (int i = top - 1; i >= 0; i--) {
        printf("-->%d", res[i]);
    }
    printf("\n");
}
int main() {
    int a[100][100], n, i, j;
    scanf("%d", &n);
    buildMatrix(a, n);
    printf("Adjacency Matrix of the graph\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++)
            printf("\t%d", a[i][j]);
        printf("\n");
    }
    printf("\nTopological order:\n");
    topological_order(n, a);
    for (i = top - 1; i >= 0; i--)
        printf("-->%d", res[i]);
    printf("\n");
```

```
    return 0;
}
```

Status : Wrong ***Marks : 0/10***

4. Problem Statement

Write a program to implement and print the topological order of a directed acyclic graph (DAG). The program should take the number of vertices and the adjacency matrix of the graph as input and then output the topological order.

When multiple vertices have zero indegree simultaneously, process them in ascending order of their vertex number (1-indexed). The output vertices are numbered starting from 1

Input Format

The first line of input consists of an integer N, representing the number of vertices in the graph.

The following N lines contain the adjacency matrix of the graph. Each line contains N space-separated integers (0 or 1), where 1 indicates a directed edge from the corresponding row vertex to the column vertex.

Output Format

The output prints the topological order of the vertices as space-separated 1-indexed integers on a single line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
1 0 0
0 1 0
1 0 0
Output: 3 1 2

Answer

```
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int adj[11][11];
    int indegree[11] = {0};
    int visited[11] = {0};
    int result[11];
    int count = 0;

    // Read matrix and calculate indegree (ignoring self-loops)
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (scanf("%d", &adj[i][j]) != 1) break;
            // Key fix: only increment indegree if i != j
            if (i != j && adj[i][j] == 1) {
                indegree[j]++;
            }
        }
    }

    // Kahn's Algorithm
    for (int step = 0; step < n; step++) {
        int u = -1;
        // Find the smallest vertex index with 0 indegree (ascending order)
        for (int i = 1; i <= n; i++) {
            if (!visited[i] && indegree[i] == 0) {
                u = i;
                break;
            }
        }

        if (u == -1) break;

        visited[u] = 1;
        result[count++] = u;

        // Update indegrees of neighbors
        for (int v = 1; v <= n; v++) {
```

```
            if (u != v && adj[u][v] == 1) {
                indegree[v]--;
            }
        }
    }

    // Print with exact spacing: "3 1 2" (no trailing space)
    for (int i = 0; i < count; i++) {
        printf("%d%s", result[i], (i == count - 1 ? "" : " "));
    }
    printf("\n");

    return 0;
}
```

Status : Partially correct ***Marks : 1/10***